DRISHTI REMOTE FLEET PLATFORM

Technical Approach Document: Zero-Root Android Fleet Operation

Architecture, Subsystem Pipelines, Device Owner Governance & Installation SOP

Drishti Systems & Embedded Engineering Team

Document Revision: 2.4 | **Target Platform:** Android 9.0 to 14+ (Unrooted Fleet)

Abstract

Commercial interactive kiosks and vending machine deployments present a fundamental infrastructure dilemma: operators require reliable, 24/7 remote device management—including interactive screen sharing, unattended gesture injection, application lifecycle control, and diagnostic shell execution—yet hardware rooting poses prohibitive security, warranty, compliance, and stability liabilities. This document details the technical architecture and operational approach of the **Drishti Remote Access Platform** tailored specifically for **non-rooted Android tablet fleets**.

By capitalizing on a single, deterministic, 10-second mounting-time USB provisioning step, the Drishti agent application is elevated to an **Android Enterprise Device Owner (DPM)**. This state fundamentally transforms the device's security model: dangerous permissions, MediaProjection background screen mirroring consent, and accessibility input dispatch are granted permanently and silently without human intervention. Screen streaming is executed via an adaptive, memory-efficient **MediaProjection/VirtualDisplay** pipeline, touch interactions are synthesized through a low-overhead **AccessibilityService** gesture injector, local debugging is enabled via an on-device embedded loopback ADB client, and all connectivity is routed through an **outbound-only reverse WebSocket tunnel** that reliably penetrates restrictive cellular firewalls and Carrier-Grade NAT (CGNAT). This approach eliminates the need for device rooting while achieving 100% autonomous reboot persistence and enterprise-grade security.

Contents

1 Executive Summary & Problem Formulation **2**
- 1.1 The Operational Context . . . 2
- 1.2 The Rooting Dilemma in Enterprise Fleets . . . 2

2 High-Level Architectural Topology **3**
- 2.1 Strict Outbound-Only Networking . . . 3

3 The Core Architectural Pillars for Non-Rooted Operation **3**
- 3.1 Pillar 1: Android Enterprise Device Owner Mode (DPM) . . . 4
  - 3.1.1 Enabling Device Owner Without Root . . . 4
  - 3.1.2 System Privileges Unlocked by Device Owner Mode . . . 4
- 3.2 Pillar 2: Unattended Screen Capture Engine (MediaProjection) . . . 4
  - 3.2.1 VirtualDisplay Configuration & Resolution Throttling . . . 5
  - 3.2.2 Buffer Stride & Row Padding Rectification . . . 5
  - 3.2.3 Decoupled Pub-Sub Transmission with Zero Idle Overhead . . . 5

3.3 Pillar 3: Remote Input Injection (AccessibilityService) 6
3.3.1 The Android Zero-Length Stroke OEM Defect Workaround 6
3.3.2 Normalized Coordinate Translation 6
3.3.3 Global Actions & Accessible Text Injection 6
3.4 Pillar 4: Embedded Local ADB Client (On-Device Wire Protocol) 7
3.5 Pillar 5: Non-Root Application Lifecycle Management 7

4 Comprehensive Device Capability Matrix **7**

5 Mounting-Time Provisioning SOP (Standard Operating Procedure) **8**

6 Network Protocol, Streaming & Scalability **8**
6.1 Head-of-Line Blocking Prevention via Decoupled Sockets 8
6.2 Bandwidth & Resource Profile 9

7 Security Architecture & Cryptographic Boundaries **9**
7.1 Transport & Mutual Authentication Security 9
7.2 Fleet Hardening via Device Owner Governance 10

8 Conclusion & Implementation Roadmap **10**

1 Executive Summary & Problem Formulation

1.1 The Operational Context

In distributed vending and automated retail networks, Android tablets function as primary customer-facing payment terminals, inventory displays, and telemetry controllers. Once mounted inside physical cabinetry, physical maintenance becomes exceptionally expensive, logistically complex, and disruptive to business operations. Technicians cannot be dispatched for routine software updates, diagnostic inspections, screen recalibrations, or application restarts.

Consequently, operators require three non-negotiable capabilities from an administrative web dashboard:

1. **Low-Latency Interactive Screen Sharing (VNC-Equivalent):** Real-time visual monitoring of tablet displays with sub-second feedback.
2. **Unattended Remote Input Injection:** Full remote touch, swipe, keyboard, and system key dispatch directly from the web canvas into running third-party kiosk applications.
3. **Remote Command & Shell Diagnostics:** The ability to inspect logs, query hardware telemetry, install/uninstall APK packages, and restart services without physical device interaction.

1.2 The Rooting Dilemma in Enterprise Fleets

Historically, remote support platforms achieved unattended device control by embedding a setuid root binary (`su`) or integrating custom OS ROMs. In a modern commercial vending fleet, rooting is fundamentally unacceptable for several critical reasons:

- **Security & Compliance Vulnerabilities:** Vending tablets frequently interface with EMV payment peripherals, PCI-DSS payment gateways, and proprietary business logic. Rooting disables Android's SELinux enforcing mode, breaks application sandboxing, and exposes the hardware to privilege escalation exploits.
- **SafetyNet & Play Integrity Failure:** Modern enterprise libraries, banking APIs, and protected media frameworks immediately refuse execution on rooted or compromised firmware.

- **OEM Warranty & Support Invalidation:** Hardware vendors repudiate manufacturer warranties and remote RMA agreements if standard system partitions are unlocked or tampered with.
- **Firmware Brittleness Across OTA Updates:** Root patches frequently fail or cause catastrophic boot-loops when minor over-the-air (OTA) system updates are deployed by tablet manufacturers.

Strategic Approach: The One-Time Physical Touch Advantage

In vending machine operations, an engineer or field technician **must physically handle every tablet exactly once** during initial machine assembly and mounting. The Drishti platform exploits this guaranteed one-time physical access: by executing a single automated provisioning script over USB before final mechanical mounting, the agent APK is enrolled as the device's **Android Device Owner**. This one-time action permanently unlocks all necessary privileged capabilities on standard, factory-sealed, non-rooted hardware.

2 High-Level Architectural Topology

The Drishti platform is partitioned into three decoupled tiers: the on-device **Android Agent**, the scalable **Relay Backend**, and the administrative **Web Dashboard**. Figure 1 illustrates the operational topology.

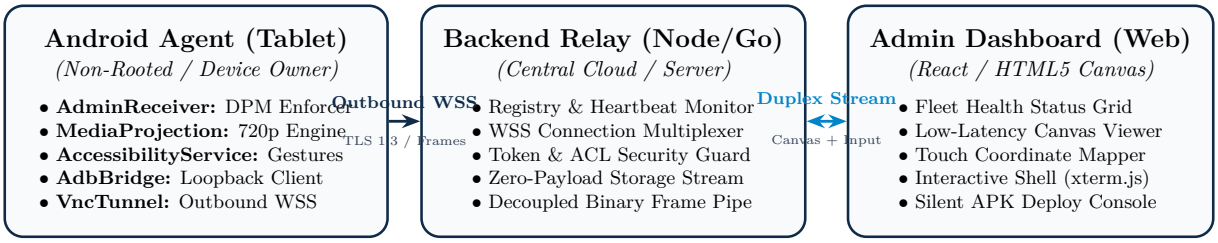


Figure 1: Drishti End-to-End System Topology for Non-Rooted Fleets.

2.1 Strict Outbound-Only Networking

Commercial vending tablets typically communicate over cellular 4G/5G modems or guest Wi-Fi networks protected by strict NAT, Carrier-Grade NAT (CGNAT), and dynamic IP allocations. Inbound TCP ports (such as standard VNC on 5900 or ADB on 5555) cannot be opened or routed to the tablet.

Drishti resolves this fundamental constraint by strictly enforcing an **outbound-only architecture**:

- The Android Agent initiates and maintains persistent outbound WebSocket connections to the central backend.
- No listening server sockets or exposed firewall rules exist on the tablet.
- Telemetry, video frames, shell streams, and synthetic touch instructions are multiplexed bidirectionally over these outbound tunnels, bypassing cellular NAT boundaries without custom VPN configurations.

3 The Core Architectural Pillars for Non-Rooted Operation

The Drishti platform replaces reliance on root privileges with four synergistic native Android framework subsystems.

3.1 Pillar 1: Android Enterprise Device Owner Mode (DPM)

Android Enterprise provides a standard administrative architecture designated as **Device Owner (DO)**. A Device Owner app is designed for institutional device management (e.g., enterprise fleets, logistics scanners, and digital signage) and possesses elevated system capabilities far beyond standard user-installed applications.

3.1.1 Enabling Device Owner Without Root

In Drishti, the agent class `com.remoteagent.AdminReceiver` extends `DeviceAdminReceiver`. During the one-time mounting phase, the provisioning script executes a single ADB command over USB:

```
adb shell dpm set-device-owner com.remoteagent/.AdminReceiver
```

Prerequisite Constraint: Clean Device State

Android OS security policies mandate that a Device Owner can only be designated when **zero user accounts (e.g., Google accounts) are enrolled on the tablet**. Consequently, provisioning must occur immediately upon initial unboxing or directly following an automated factory reset.

3.1.2 System Privileges Unlocked by Device Owner Mode

Once established as Device Owner, the Drishti Agent programmatically exercises administrative authority via `DevicePolicyManager`:

1. **Suppression of User Confirmation Dialogs:** Standard Android prompts for background activity starts, permission elevations, and system setting modifications are pre-empted and automatically approved.
2. **Silent Permission Auto-Granting:** The agent grants itself sensitive runtime permissions without presenting user prompts.
3. **Silent Accessibility Service Activation:** Standard users must navigate nested Settings menus to enable Accessibility services. Device Owner policies allow the agent to register and maintain its accessibility input injection engine silently.
4. **Programmatic Wireless ADB Configuration:** The agent can configure `Settings.Global.ADB_ENABLED` and manage network debugging parameters programmatically.
5. **Tamper-Proof Reboot Persistence:** Device Owner status is permanently recorded in the hardware's encrypted system partition (`/data/system/device_owner_2.xml`). It cannot be uninstalled or revoked by an interactive user and automatically resumes on tablet boot via `BootReceiver`.

3.2 Pillar 2: Unattended Screen Capture Engine (MediaProjection)

Standard VNC servers capture screens by directly reading the Linux framebuffer (`/dev/graphics/fb0`) or querying the SurfaceFlinger display compositor—operations strictly prohibited without root. Drishti utilizes the native Android **MediaProjection API** integrated into a dedicated foreground service (`MediaProjectionCaptureService`).

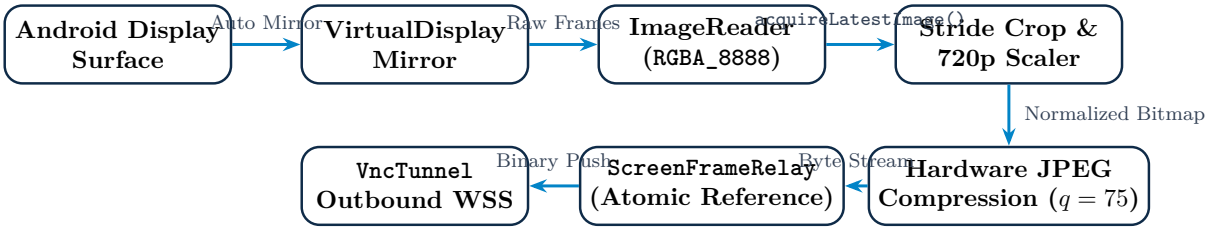


Figure 2: Drishti Zero-Root Screen Capture and Frame Relay Pipeline.

3.2.1 VirtualDisplay Configuration & Resolution Throttling

To guarantee fluid frame transmission over bandwidth-constrained cellular links, `MediaProjectionCaptureServic` reads real display dimensions and scales the output surface dynamically:

```
// Cap resolution to 720p maximum width while preserving exact aspect ratio
val scale = minOf(1.0f, MAX_WIDTH.toFloat() / metrics.widthPixels)
screenWidth = (metrics.widthPixels * scale).toInt()
screenHeight = (metrics.heightPixels * scale).toInt()

val reader = ImageReader.newInstance(
    screenWidth, screenHeight,
    PixelFormat.RGBA_8888, 2
)
virtualDisplay = mediaProjection?.createVirtualDisplay(
    "DrishtiCapture", screenWidth, screenHeight, screenDensity,
    DisplayManager.VIRTUAL_DISPLAY_FLAG_AUTO_MIRROR,
    reader.surface, null, handler
)
```

3.2.2 Buffer Stride & Row Padding Rectification

Hardware image planes require 64-byte row alignments. Drishti explicitly detects and removes row padding before compression to avoid distorted or skewed frames:

```
val planes = image.planes
val buffer = planes[0].buffer
val pixelStride = planes[0].pixelStride
val rowStride = planes[0].rowStride
val rowPadding = rowStride - pixelStride * screenWidth

val bmp = Bitmap.createBitmap(
    screenWidth + rowPadding / pixelStride, screenHeight,
    Bitmap.Config.ARGB_8888
)
bmp.copyPixelsFromBuffer(buffer)

// Extract pure active screen bounds if padding exists
val cropped = if (rowPadding > 0) {
    Bitmap.createBitmap(bmp, 0, 0, screenWidth, screenHeight).also { bmp.recycle()
        }
} else {
    bmp
}
```

3.2.3 Decoupled Pub-Sub Transmission with Zero Idle Overhead

Continuous screen capture on unmanned tablets wastes CPU cycles and generates excessive cellular data traffic. Drishti introduces an in-memory lock-free frame hub (`ScreenFrameRelay`). When no administrator is actively viewing the device in the dashboard:

1. The backend issues a `viewer_disconnected` control frame.
2. `VncTunnel` unregisters its listener from `ScreenFrameRelay`.
3. Frame encoding and transmission immediately cease, reducing idle cellular data consumption to sub-kilobyte heartbeat levels.

3.3 Pillar 3: Remote Input Injection (AccessibilityService)

Traditional remote control software injects synthetic touch events by issuing shell commands to `/dev/input/event*` or executing `input tap x y` via root. On non-rooted devices, these operations are blocked by SELinux and permission gates.

Drishti utilizes the public Android **AccessibilityService API** (`DrishtiAccessibilityService`), which natively supports programmatic gesture dispatching (`dispatchGesture()`) across the entire OS without root.

3.3.1 The Android Zero-Length Stroke OEM Defect Workaround

On multiple OEM Android distributions common in kiosk tablets, an input path consisting solely of `moveTo(px, py)` is rejected as empty by the system gesture engine. Drishti circumvents this platform bug by programmatically generating a micro-vector delta:

```
fun injectTapPixels(px: Float, py: Float) {
    // Construct a micro-vector path to satisfy Android OEM stroke validators
    val path = Path().apply {
        moveTo(px, py)
        lineTo(px + 0.5f, py + 0.5f)
    }
    val gesture = GestureDescription.Builder()
        .addStroke(GestureDescription.StrokeDescription(path, 0L, 60L))
        .build()

    dispatchGesture(gesture, object : GestureResultCallback() {
        override fun onCompleted(gestureDescription: GestureDescription?) {
            Log.i(TAG, "Tap completed at ($px, $py)")
        }
        override fun onCancelled(gestureDescription: GestureDescription?) {
            Log.w(TAG, "Tap cancelled by OS window manager at ($px, $py)")
        }
    }, null)
}
```

3.3.2 Normalized Coordinate Translation

The web dashboard canvas translates mouse clicks and drag vectors into normalized floating-point coordinates $(nx, ny \in [0.0, 1.0])$. When received by `VncTunnel`, `DrishtiAccessibilityService` calculates absolute hardware screen pixels using current `DisplayMetrics`:

$$px = \text{clamp}\left(nx \times W_{\text{real}},\, 0,\, W_{\text{real}} - 1\right), \quad py = \text{clamp}\left(ny \times H_{\text{real}},\, 0,\, H_{\text{real}} - 1\right) \tag{1}$$

This guarantees pixel-perfect touch alignment across varying tablet aspect ratios and display orientations.

3.3.3 Global Actions & Accessible Text Injection

In addition to touch gestures, the accessibility layer provides system-wide hardware button emulation:

- **Global Navigation:** Direct dispatch of `GLOBAL_ACTION_BACK`, `GLOBAL_ACTION_HOME`, and `GLOBAL_ACTION_RE`

- **Text Input Dispatch:** The agent locates the active focused input node via `findFocus(AccessibilityNodeIr` and commits the text string directly using `ACTION_SET_TEXT`. If an active accessibility node is unavailable, it automatically stages the string to the Android system clipboard (`ClipboardManager`).

3.4 Pillar 4: Embedded Local ADB Client (On-Device Wire Protocol)

While screen sharing and input injection provide comprehensive visual control, technical diagnostics frequently require shell-level access.

How ADB Operates: Local Loopback Architecture

ADB is not a monolithic binary; it is a networked client/server protocol. The Android system daemon (`adbd`) listens on local TCP socket port `5555`. A client communicates using standard wire-protocol packets.
On a non-rooted device, an app cannot replace `adbd`, but it can act as an **ADB Client**. Drishti embeds a pure Kotlin/Java wire-protocol ADB implementation (`AdbLib`). The agent connects locally via loopback (`127.0.0.1:5555`), performs standard RSA key exchange, and exposes interactive shell sessions streamed over the outbound WebSocket.

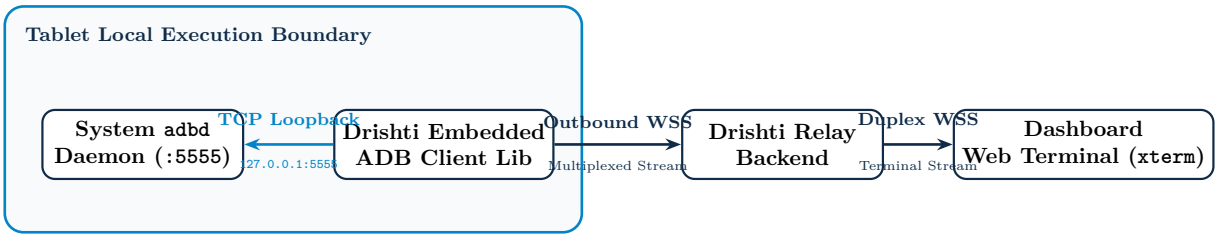


Figure 3: Embedded On-Device ADB Client Loopback Architecture.

3.5 Pillar 5: Non-Root Application Lifecycle Management

In `CommandExecutor.kt`, the agent handles application execution, termination, and updates within non-root constraints:

- **Silent Remote APK Updates:** Because the agent is Device Owner, it can pass downloaded update packages directly to `android.content.pm.PackageInstaller`. The installation session is committed silently in the background without user interaction.
- **Reliable Kiosk App Launching:** To ensure mission-critical kiosk software remains active, `CommandExecutor` resolves the target package's launchable intent via `packageManager.getLaunchIntentForPa` If the OEM launcher component is masked, it leverages the native Android `monkey` runner (`monkey -p <pkg> -c android.intent.category.LAUNCHER 1`).
- **Graceful Application Dismissal:** Non-rooted apps cannot execute `am force-stop` on third-party packages. Drishti resolves this by having `DrishtiAccessibilityService` inject an immediate `GLOBAL_ACTION_HOME` command, followed by `ActivityManager.killBackgroundProcesses()`, cleanly dismissing the foreground app without system instability.

4 Comprehensive Device Capability Matrix

Table 1 contrasts the operational capabilities, setup requirements, and security profiles across the three device tiers.

Capability / Dimension	Rooted Fleet	Drishti Device Owner (Standard Approach)	Stock Non-Rooted (Unprovisioned Fallback)
Setup Requirement	Unlocking bootloader; flashing custom su	One USB command at mounting (10 seconds)	Manual user toggle in system Settings
Interactive Screen Share	Framebuffer / screencap	**MediaProjection (Silent consent via DPM)**	MediaProjection (Requires physical consent popup)
Touch / Gesture Injection	Direct `/dev/input` writes	**AccessibilityService (`dispatchGesture`)**	AccessibilityService (Manual toggle required)
Remote Shell Execution	Full root bash (`su`)	**Local ADB loopback client + Sandboxed sh**	Sandboxed sh / temporary Wireless ADB
Reboot Survival	Yes	**Yes (100% automated via DPM & BootReceiver)**	Fragile (OEMs frequently reset developer toggles)
Silent APK Installation	`pm install` via root	**PackageInstaller session API via DPM**	Launches system prompt dialog
Security Posture	Severely compromised (SELinux bypassed)	**Strict Enterprise Grade (SELinux Enforcing)**	Standard Consumer Sandbox
Commercial / Banking Feasibility	Non-compliant (Fails SafetyNet/PCI-DSS)	**Fully Compliant (Standard Android Enterprise)**	Compliant

Table 1: Comparison Matrix: Rooted vs. Drishti Device Owner vs. Unprovisioned Stock.

5 Mounting-Time Provisioning SOP (Standard Operating Procedure)

To operationalize this approach, technicians follow a concise 4-step checklist before mounting tablets into vending machines. The process is fully automated via `provision_tablet.sh`.

6 Network Protocol, Streaming & Scalability

6.1 Head-of-Line Blocking Prevention via Decoupled Sockets

Transmitting large binary video frame bursts over the same TCP/WebSocket channel as latency-sensitive control commands causes Head-of-Line (HoL) blocking: an interactive keystroke or tap event can be delayed by hundreds of milliseconds while a heavy 80 KB video frame buffer finishes transmission.

Drishti implements a dual-socket decoupled channel architecture:

1. **Control Channel (JSON text stream):** Dedicated to telemetry, heartbeat, shell I/O, and remote command dispatch. Operates with `TCP_NODELAY` enabled to ensure sub-30ms round-trip latency.

2. **Screen Frame Channel (Binary stream):** A dedicated WebSocket channel instantiated on demand per active screen-sharing session. Frames are dispatched as raw binary JPEG buffers directly from `ScreenFrameRelay`.

Standard Operating Procedure Checklist (Duration: ≈ 10 seconds per tablet)

1. **Unboxing / Factory State Verification:** Ensure the tablet is clean or factory-reset. **Do not sign into any Google or enterprise accounts.** Connect the tablet to the technician's workstation via USB.
2. **Enable Developer Debugging:** Navigate to `Settings` → `About Tablet`. Tap `Build Number` 7 times to reveal Developer Options. Enable `USB Debugging`.
3. **Execute Automated Provisioning Script:** From the technician's laptop, execute the deployment script:

```
bash provision_tablet.sh
```

   The script verifies ADB communication, validates zero enrolled accounts, sideloads the Drishti Agent APK, and assigns Device Owner privileges:

```
adb shell dpm set-device-owner com.remoteagent/.AdminReceiver
```

4. **Mount Tablet into Machine:** Disconnect the USB cable. Install the tablet into the vending machine frame. Upon initial network connection, the agent dials the backend relay and registers online automatically.

6.2 Bandwidth & Resource Profile

Table 2 details the network throughput and system resource consumption across device operating states.

Operating State	Network Throughput	CPU Utilization	Memory Footprint
Standby / Heartbeat Only	< 0.5 KB/sec	$< 0.8\%$ (Idle thread)	≈ 38 MB
Interactive Shell Session	$1.0 - 5.0$ KB/sec	$1.5 - 3.0\%$	≈ 42 MB
Active Screen Share (15 FPS, 720p)	$350 - 650$ KB/sec	$8.0 - 14.0\%$ (GPU/Media-Codec)	≈ 64 MB

Table 2: Resource and Bandwidth Profiling on Typical Quad-Core ARM Cortex-A55 Tablet.

7 Security Architecture & Cryptographic Boundaries

7.1 Transport & Mutual Authentication Security

- **TLS 1.3 Encryption:** All transport between tablets, backend relays, and administrative dashboards is encrypted via TLS 1.3 with modern cipher suites (AES-256-GCM / ChaCha20-Poly1305).
- **Cryptographic Device Tokens:** Each tablet is provisioned with a unique, high-entropy API key passed via the HTTP header `x-agent-key` during WebSocket handshakes.
- **Zero Persistent Storage of Visual Frames:** To satisfy privacy standards and prevent data exfiltration, screen frames are never written to disk or recorded on backend relay servers. Frames reside strictly in transient RAM buffers and are immediately overwritten upon transmission.

7.2 Fleet Hardening via Device Owner Governance

Because the Drishti Agent is enrolled as Device Owner, it can enforce device hardening policies on the vending tablet:

- **Lock Task Mode (Kiosk Pinning):** The agent pins the customer-facing vending application to the foreground, suppressing the system notification shade, navigation bars, and hardware key shortcuts.
- **Uninstall Prevention:** Device Owner status prevents unauthorized users or malware from uninstalling the agent application via standard system settings.

8 Conclusion & Implementation Roadmap

The technical approach detailed in this document provides a robust, enterprise-compliant foundation for remote Android fleet management without device rooting:

- **Root-Free Security:** Eliminates all security, compliance, warranty, and OTA stability liabilities inherent to custom ROMs and `su` exploits.
- **Deterministic One-Time Setup:** Capitalizes on the standard 10-second mounting-time USB connection to establish permanent Device Owner administrative authority.
- **Full Interactive Control:** Delivers low-latency screen sharing (MediaProjection), unattended touch injection (AccessibilityService), and remote diagnostics (Embedded ADB Client) over outbound-only reverse WebSocket connections.
- **Autonomous Reboot Persistence:** Automatically recovers and resumes operations across unexpected power cycles without human intervention.

Fleet Engineering Roadmap

- **Phase 1 (Current Production):** MediaProjection JPEG frame capture, AccessibilityService gesture injection, and Device Owner enrollment via `provision_tablet.sh`.
- **Phase 2 (Q3 Enhancement):** Migration of the video pipeline to hardware-accelerated H.264/WebRTC peer-to-peer data channels to reduce backend relay server bandwidth by up to 60%.
- **Phase 3 (Enterprise Scale):** Integration of zero-touch automated QR enrollment to eliminate USB manual intervention during initial factory unboxing.